

SKALA · SK AI Leader Academy — 백엔드 개발 실습

오늘 뭐 먹지?

Spring Boot 메뉴 추천 API 실습
실습과제 구현 · 자체 개선 · Docker 컨테이너화 리포트

과목
백엔드 2일차 실습

작성일
2026-08-04

실행 환경
**Spring Boot 3.5.16 / JDK 21 /
Docker**

작성자
서혁인

프로젝트
**springboot-menu-
recommendation**

제출 형식
PDF (Slack 댓글 첨부)

본 문서는 로컬 터미널 + Docker Desktop 환경에서 직접 빌드/실행/검증한 결과를 기반으로 작성되었습니다.

오늘 뭐 먹지? — Spring Boot 메뉴 추천 실습 보고서

프로젝트	springboot-menu-recommendation (GitHub: tjgurdls9/skala-SpringPractice)
과제 내용	기존에 구현된 기본 메뉴 추천 기능 위에, 날씨별/기분별/가격대별/나만의 추천 4개 API를 직접 구현하고, 프로젝트 루트에 Dockerfile을 작성해 컨테이너로 빌드/실행
제출 조건	평가 X, 제출 O
사용 기술	Spring Boot, Gradle, Docker(멀티스테이지 빌드), Docker Scout
작성자	서혁인

목차

- 1. 기존에 구현되어 있던 기능
- 2. 실습과제로 직접 구현한 API (4개)
- 3. 나만의 차별화 포인트 (직접 개선한 부분)
- 4. Docker 빌드 & 실행 검증
- 5. Docker Scout 취약점 개선 (Before / After)

1. 기존에 구현되어 있던 기능

실습 시작 시점에 이미 만들어져 있던, Component Scan / 생성자 주입 / @RestController 학습용 API

API	설명
GET /api/hello/{name}	이름을 받아 인사말 반환
GET /api/menu	고정된 기본 추천
GET /api/menu/random	메뉴 목록 중 무작위 추천
GET /api/menu/{category}	korean / chinese / japanese / snack 카테고리별 추천
GET /api/menu/json/{category}	카테고리별 추천을 JSON(MenuResponse)으로 응답

MenuService Bean을 MenuController 가 생성자 주입으로 사용하는 구조를 학습했습니다.

2. 실습과제로 직접 구현한 API (4개)

요구사항: 화면의 주황색 버튼 4개가 부르는 주소에 맞춰 Controller/Service에 로직을 직접 작성

과제	API	로직
① 날씨 별 추천	GET /api/menu/weather/{weather}	sunny / rainy / hot / cold > switch 분기
② 기분 별 추천	GET /api/menu/mood/{mood}	happy / sad / tired / stressed > switch 분기
③ 가격 대별 추 천	GET /api/menu/price/search? min=&max=	@RequestParam 으로 두 값을 받아 구간별 추천
④ 나만 의 추천	GET /api/menu/my/{companion}	solo / friends / family (같이 먹는 사람 기 준, 자유 설계)

기존 `recommendByCategory` / `menuByCategory` 와 동일한 구조(Controller > Service,
`@PathVariable` / `@RequestParam`, switch 분기)를 그대로 반복 적용해서 만들었습니다.

3. 나만의 차별화 포인트 (직접 개선한 부분)

과제 안내문: "여기에 자신만의 차별화 포인트를 반영해 보시거나, 자신만의 새로운 아이템으로 반영하셔도 됩니다" — 이 부분을 실제로 반영한 내역

4개 API 구현이라는 최소 요구사항을 채운 뒤, 요구되지 않은 부분까지 스스로 점검하며 아래 2가지를 **직접 찾아서 개선**했습니다. 특히 ①번은 단순 버그 수정이 아니라, "실습 화면이 항상 같은 값만 테스트한다"는 구조적 한계를 발견하고 고친 것이라 가장 신경 쓴 차별화 포인트입니다.

항목	Before	After
① index.html 하드코딩 제거	버튼이 <code>rainy / happy / min=5000&max=10000 / solo</code> 등 고정값 하나만 호출 > 매번 눌러도 같은 결과만 확인 가능	클릭할 때마다 유효한 값 후보 중 하나를 무작위로 골라 호출 > 반복 클릭만으로 각 API의 switch 분기(4가지 날씨, 4가지 기분 등)를 전부 검증할 수 있는 테스트 화면으로 발전
② 개발 편의성 패치	<code>bootRun</code> 후 매번 브라우저 직접 열어야 함	서버가 응답 가능해지는 순간 자동으로 기본 브라우저에서 화면을 열도록 <code>build.gradle</code> 에 로직 추가 (macOS <code>open</code> 명령 사용)

요구사항(4개 API 구현)만 채우고 끝내지 않고, 실제로 화면을 눌러보며 "이 버튼은 항상 같은 값만 부른다" 같은 문제를 스스로 발견해서 고친 부분을 이번 실습의 차별화 포인트로 제출합니다.

실제 동작 화면 — 같은 버튼, 다른 결과

"① 날씨별 추천" 버튼을 **연속으로 두 번** 눌렀을 때의 화면입니다. 하드코딩이었다면 매번 `rainy`로 고정되어 같은 응답만 나왔겠지만, 랜덤 호출로 바꾼 뒤에는 클릭할 때마다 다른 날씨 값이 호출되어 다른 메뉴가 나오는 것을 확인할 수 있습니다.

오늘 뭐 먹지?

버튼을 누르면 Spring Boot REST API의 응답을 확인할 수 있습니다.

기본 기능

이미 만들어져 있는 API입니다. 눌러서 동작을 확인해 보세요.

기본 추천

랜덤 추천

한식 추천

중식 추천

일식 추천

JSON 응답

실습과제 직접 만들기

아래 버튼은 화면에만 있고 서버에는 아직 API가 없습니다. MenuService와 MenuController에 직접 코드를 작성해서 버튼이 동작하게 만들어 보세요.

① 날씨별 추천

② 기분별 추천

③ 가격대별 추천

④ 나만의 추천

hot에 어울리는 메뉴는 냉면입니다.

1번째 클릭 — 무작위로 `hot` 이 호출되어 "hot에 어울리는 메뉴는 냉면입니다." 응답

오늘 뭐 먹지?

버튼을 누르면 Spring Boot REST API의 응답을 확인할 수 있습니다.

기본 기능

이미 만들어져 있는 API입니다. 눌러서 동작을 확인해 보세요.

기본 추천

랜덤 추천

한식 추천

중식 추천

일식 추천

JSON 응답

실습과제 직접 만들기

아래 버튼은 화면에만 있고 서버에는 아직 API가 없습니다. MenuService와 MenuController에 직접 코드를 작성해서 버튼이 동작하게 만들어 보세요.

① 날씨별 추천

② 기분별 추천

③ 가격대별 추천

④ 나만의 추천

sunny에 어울리는 메뉴는 타코 플래터입니다.

2번째 클릭 — 같은 버튼인데 무작위로 **sunny**가 호출되어 "sunny에 어울리는 메뉴는 타코 플래터입니다." 응답

4. Docker 빌드 & 실행 검증

요구사항: 프로젝트 루트에 Dockerfile 생성 및 터미널에서 docker build 명령어 사용

멀티스테이지 빌드로 **Dockerfile** 을 작성했습니다 — 1단계(**eclipse-temurin:21-jdk**)에서 Gradle Wrapper로 **bootJar** 를 실행하고, 2단계(**eclipse-temurin:21-jre-alpine**)에 결과 jar만 복사해서 최종 이미지 용량을 줄였습니다.

```
docker build -t springboot-menu-recommendation .
docker run -p 8080:8080 springboot-menu-recommendation
```

```
> springboot-menu-recommendation git:(main) x docker build -t springboot-menu-recommendation .
[+] Building 1.4s (18/18) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 564B                                0.0s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine 1.3s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk      1.3s
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io        0.0s
=> [internal] load .dockerignore                                       0.0s
=> => transferring context: 81B                                          0.0s
=> [build 1/8] FROM docker.io/library/eclipse-temurin:21-jdk@sha256:b388ef0a5da6bf6a265f22a3b298 0.0s
=> => resolve docker.io/library/eclipse-temurin:21-jdk@sha256:b388ef0a5da6bf6a265f22a3b298e7a330 0.0s
=> [internal] load build context                                       0.0s
=> => transferring context: 1.43kB                                       0.0s
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:3f08b13888f595cc49e 0.0s
=> => resolve docker.io/library/eclipse-temurin:21-jre-alpine@sha256:3f08b13888f595cc49edabea725 0.0s
=> CACHED [stage-1 2/3] WORKDIR /app                                    0.0s
=> CACHED [build 2/8] WORKDIR /app                                    0.0s
=> CACHED [build 3/8] COPY gradlew .                                    0.0s
=> CACHED [build 4/8] COPY gradle gradle                              0.0s
=> CACHED [build 5/8] COPY build.gradle settings.gradle ./           0.0s
=> CACHED [build 6/8] RUN chmod +x gradlew                            0.0s
=> CACHED [build 7/8] COPY src src                                    0.0s
=> CACHED [build 8/8] RUN ./gradlew bootJar --no-daemon               0.0s
=> CACHED [stage-1 3/3] COPY --from=build /app/build/libs/*.jar app.jar 0.0s
=> exporting to image                                                 0.0s
=> => exporting layers                                                  0.0s
=> => exporting manifest sha256:bee924015bbc440096bb477e97bc2502063dd5f4c88687c1abfa914fb1456be2 0.0s
=> => exporting config sha256:a21bdb6fffe893a8cbc03c3d0d01dd59bada198e9600d919ff7e90f71d342af1 0.0s
=> => exporting attestation manifest sha256:54faa74bd5a5eb043bb1064ffb7239d27abb03956cdb7c51b6b2 0.0s
=> => exporting manifest list sha256:3a755aaa527be4af937575de48175a9ded43074e0b969b987e4185ad524 0.0s
=> => naming to docker.io/library/springboot-menu-recommendation:latest 0.0s
=> => unpacking to docker.io/library/springboot-menu-recommendation:latest 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/grgmlk6ccdgy70gkwvgbuw3uf
```

docker build 실행 결과 — 18/18 스테이지 전부 FINISHED, 이미지 springboot-menu-recommendation:latest 생성 완료

```
➤ springboot-menu-recommendation git:(main) x docker run -p 8080:8080 springboot-menu-recommendation
```

[illegible]

```
:: Spring Boot ::                (v3.5.16)
```

```
2026-08-04T07:15:26.993Z INFO 1 --- [springboot-menu-recommendation] [      main] com.example.menu
.MenuApplication      : Starting MenuApplication v0.0.1-SNAPSHOT using Java 21.0.11 with PID 1 (/app/
app.jar started by root in /app)
2026-08-04T07:15:26.994Z INFO 1 --- [springboot-menu-recommendation] [      main] com.example.menu
.MenuApplication      : No active profile set, falling back to 1 default profile: "default"
2026-08-04T07:15:27.329Z INFO 1 --- [springboot-menu-recommendation] [      main] o.s.b.w.embedded
.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-08-04T07:15:27.335Z INFO 1 --- [springboot-menu-recommendation] [      main] o.apache.catalin
a.core.StandardService : Starting service [Tomcat]
2026-08-04T07:15:27.335Z INFO 1 --- [springboot-menu-recommendation] [      main] o.apache.catalin
a.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.55]
2026-08-04T07:15:27.346Z INFO 1 --- [springboot-menu-recommendation] [      main] o.a.c.c.C.[Tomca
t].[localhost].[/]    : Initializing Spring embedded WebApplicationContext
2026-08-04T07:15:27.347Z INFO 1 --- [springboot-menu-recommendation] [      main] w.s.c.ServletWeb
ServerApplicationContext : Root WebApplicationContext: initialization completed in 330 ms
2026-08-04T07:15:27.398Z INFO 1 --- [springboot-menu-recommendation] [      main] o.s.b.a.w.s.Welc
omePageHandlerMapping : Adding welcome page: class path resource [static/index.html]
2026-08-04T07:15:27.529Z INFO 1 --- [springboot-menu-recommendation] [      main] o.s.b.w.embedded
.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-08-04T07:15:27.544Z INFO 1 --- [springboot-menu-recommendation] [      main] com.example.menu
.MenuApplication      : Started MenuApplication in 0.721 seconds (process running for 0.977)
```

docker run -p 8080:8080 실행 결과 — Spring Boot 3.5.16, Tomcat 8080 포트 초기화, "Started MenuApplication"

로그로 정상 기동 확인

```

→ springboot-menu-recommendation git:(main) x curl http://localhost:8080/api/menu/weather/rainy
rainy에 어울리는 메뉴는 해물파전입니다.%
→ springboot-menu-recommendation git:(main) x curl http://localhost:8080/api/menu/mood/happy
오늘 기분엔 치킨 어떠세요?%
→ springboot-menu-recommendation git:(main) x curl "http://localhost:8080/api/menu/price/search?min=5000&max=10000"
5000원~10000원 사이 추천 메뉴는 돈가스입니다.%
→ springboot-menu-recommendation git:(main) x curl http://localhost:8080/api/menu/my/solo
solo이라면 편의점 도시락 어떠세요?%
→ springboot-menu-recommendation git:(main) x

```

컨테이너 실행 중 curl로 날씨별/기분별/가격대별/나만의 추천 4개 API 호출 — 전부 정상 응답 확인

5. Docker Scout 취약점 개선 (Before / After)

Docker Desktop이 이미지를 자동 스캔해 알려준 보안 취약점(CVE)을 원인 분석 후 개선

원인 파악: 취약점 대부분이 OS 베이스 이미지가 아니라, 지원 종료된 **Spring Boot 3.4.5** 프레임워크 의존성(2025-12-31 OSS 지원 종료)에서 나온다는 것을 확인했습니다.

조치: ① 베이스 이미지를 `eclipse-temurin:21-jre` > `eclipse-temurin:21-jre-alpine` 으로 변경,
② Spring Boot 버전을 `3.4.5` > `3.5.16` (지원 중)으로 업그레이드

The screenshot shows the Docker Desktop interface with the 'Images' tab selected. The image 'springboot-menu-recommendation:latest' is highlighted. The 'Layers' section shows the image structure. The 'Vulnerabilities' section shows 58 vulnerabilities found in the image. The 'Packages' section shows 244 packages scanned. The interface includes a sidebar with navigation options like 'Gordon', 'Containers', 'Images', 'Logs', 'Volumes', 'Kubernetes', 'Builds', 'Models', 'MCP Toolkit', 'Docker Hub', and 'Extensions'. The bottom status bar shows 'Engine running', 'RAM 2.24 GB', 'CPU 0.10%', and 'Disk: 2.35 GB used (limit 910.69 GB)'.

Layer	Operation	Size
16	WORKDIR /app	8.19 KB
17	COPY /app/build/libs/*.jar a...	20.77 MB
18	EXPOSE [8080/tcp]	0 B
19	ENTRYPOINT ["java" "-jar" "a...	0 B

CVE ID	Severity	Fixable
CVE-2026-41293	9.8 C	✓
CVE-2026-43512	9.8 C	✓
CVE-2026-29145	9.1 C	✓
CVE-2026-43515	9.1 C	✓
CVE-2025-48988	8.7 H	✓

[BEFORE] Docker Desktop Images 탭 — Vulnerabilities 58건, 244개 패키지 스캔 (jre 베이스 + Spring Boot 3.4.5)

```

→ springboot-menu-recommendation git:(main) ✕ docker scout quickview springboot-menu-recommendation
✓ Image stored for indexing
✓ Indexed 122 packages
✓ Provenance obtained from attestation
✓ Policy evaluation completed

i Base image was auto-detected. To get more accurate results, build images with max-mode provenance
attestations.
Review docs.docker.com ↗ for more information.

Target          | springboot-menu-recommendation:latest | 0C  2H  8M  1L  12?
digest          | 3a755aaa527b |
Base image      | eclipse-temurin:21-jre-alpine | 0C  2H  5M  1L  12?
Updated base image | eclipse-temurin:26-jre-alpine | 0C  0H  4M  0L  12?
                                     -2   -1   -1

Policy status FAILED (4/7 policies met)
Health score B (72%)



| Status | Policy                                      | Results        |
|--------|---------------------------------------------|----------------|
| !      | Image runs as the root user                 |                |
| !      | Copyleft licensed packages found            | 59 packages    |
| ✓      | No fixable critical or high vulnerabilities | 0C  0H  0M  0L |
| ✓      | No high-profile vulnerabilities             | 0C  0H  0M  0L |
| ✓      | No outdated base images                     |                |
| ✓      | No unapproved base images                   | 0 deviations   |
| !      | Required supply chain attestations missing  | 1 deviation    |



What's next:
View policy violations → docker scout policy springboot-menu-recommendation
View vulnerabilities → docker scout cves springboot-menu-recommendation
View base image update recommendations → docker scout recommendations springboot-menu-recommendation
Compare with the latest in the registry → docker scout compare --to-latest springboot-menu-recommendation

```

[AFTER] docker scout quickview 실행 결과 — Health score B(72%), "No fixable critical or high vulnerabilities" 정책 통과

항목	Before	After
Critical	4	0
High	22	2
Medium	27	8
Low	13	1
Health score	D (50%)	B (72%)
이미지 용량	약 141MB	약 92MB

베이스 이미지 교체는 이미지 용량 감소에는 효과가 있었지만, 실질적인 취약점 감소는 **지원 종료된 프레임워크 버전을 최신 버전으로 올리는 것**에서 나왔습니다. 버전 업그레이드 후 6개 API를 재검증해서 정상 동작을 확인했습니다.